analysis_8.8.26

Shawn Wang

2026-09-05

semantic distance processing

```
monologue_clean <- clean_monologue_or_list(dat = processed_df, wordcol = "words", omit_stops = TRUE, le
dists_ngram2word <- dist_ngram2word(dat = monologue_clean, ngram = 2)

# dists_ngram2word %>%
#   group_by(ID) %>%
#   mutate(word_num = row_number()) %>%
#   ggplot(mapping = aes(x = word_num, y = CosDist_2gram_glo, group = 1)) +
#   geom_line(stat = "summary") +
#   geom_smooth()



# dists_ngram2ngram <- dist_ngram2ngram(dat = monologue_clean, ngram = 2)
# dists_ngram2ngram %>%
#   group_by(ID) %>%
#   mutate(word_num = row_number()) %>%
#   ggplot(mapping = aes(x = word_num, y = CosDist_2gram_GLO, group = 1)) +
#   geom_line(stat = "summary") +
#   geom_smooth()
```

```
sentiments <- c("emo_anger", "emo_anxiety", "emo_arousal_b24", "emo_excitement",
                "emo_happiness", "emo_intensity", "emo_sadness", "emo_valence_b24")

#yoke sentiment values
speech_df_sent <- speech_df %>%
  left_join(y = lookup_Jul25, by = c("word_clean" = "word")) %>%
  select(duration:id_row_postsplit, all_of(sentiments))

speech_sent_sum <- speech_df_sent %>%
  group_by(ID) %>%
  summarize(ID = unique(ID),
            duration = unique(duration),
            across(.cols = emo_anger:emo_valence_b24, .fns = ~ mean(.x, na.rm = TRUE), .names = "{.col}
  left_join(cort_raw, by = "ID") %>%
  mutate(Date = TSSTDate)
```

```
joined_data <- merge(x = speech_sent_sum, y = data_raw, by = c("ID", "Date")) %>%
  filter(ID != 9736)
```

set up linguistic terms

```
norms <- c("emo_anger","emo_anxiety","emo_arousal_b24","emo_boredom","emo_confusion",
  "emo_excitement","emo_guilt","emo_happiness","emo_intensity","emo_sadness","emo_trust",
  "emo_valence_b24","lex_AoA","lex_freqlg10","lex_n_morphemes","lex_n_senses","phon_n_lett",
  "phon_nsyll","sem_auditory","sem_cnc_b24","sem_diversity","sem_neighbors","sem_visual")

word_norms <- speech_df %>%
  filter(!is.na(word_clean)) %>%
  left_join(lookup_Jul25 %>% select(word, all_of(norms)), by = c("word_clean" = "word")) %>%
  group_by(ID) %>%
  summarize(across(all_of(norms), ~ mean(.x, na.rm = TRUE)), .groups = "drop")

word_count <- speech_df %>%
  group_by(ID) %>%
  summarize(n_words = n())

# word_coherence <- dists_ngram2word %>%
#   group_by(ID) %>%
#   summarize(n2w_glo_mean = mean(CosDist_2gram_glo, na.rm = TRUE))
```

set up the database

```
db <- data_raw %>%
  mutate(Date = as.Date(Date))

tsst_dates <- read_csv(file = "raw_data/tsst_dates.csv") %>%
  mutate(ID = as.character(ID))
```

```
## Rows: 91 Columns: 3
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (1): date_src
## dbl  (1): ID
## date (1): Date
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
speech_visit <- word_norms %>%
  inner_join(tsst_dates, by = "ID") %>%
  inner_join(db %>% select(ID, Date, ChildBirthday, ChildGender, ChildRace), by = c("ID", "Date")) %>%
  # filter(!ID %in% c("9736", "9730")) %>%
  #filter(!ID %in% c("9736")) %>%
  mutate(age = as.numeric(Date - as.Date(ChildBirthday)) / 365,
         female = ChildGender) %>%
  select(-c(ChildBirthday, ChildGender, date_src)) %>%
  left_join(word_count, by = "ID") %>%
  mutate(ChildRace = as.factor(ChildRace))

add_prior <- function(dat, v, win = 365) {
  prior <- db %>% select(ID, wave_date = Date, val = all_of(v)) %>% drop_na() %>%
    inner_join(dat %>% select(ID, Date), by = "ID") %>%
```

```
    filter(wave_date <= Date, wave_date >= Date - win) %>%
    group_by(ID) %>% slice_max(wave_date, n = 1, with_ties = FALSE) %>% ungroup() %>%
    transmute(ID, "{v}" := val, "{v}_gap" := as.numeric(Date - wave_date))
  left_join(dat, prior, by = "ID")
}
#
# analysis_df <- speech_visit %>% add_prior("CHS_TOT") %>% add_prior("CDI") %>% add_prior("MASC") %>%
#   relocate(ID, Date)


# CDI/MASC recorded AT the speech visit -- exact ID + Date match
add_sameday <- function(dat, v) {
  sameday <- db %>%
    select(ID, Date, val = all_of(v)) %>% drop_na() %>%
    transmute(ID, Date, "{v}" := val, "{v}_gap" := 0)
  left_join(dat, sameday, by = c("ID", "Date"))
}

IVs <- c("CHS_TOT", "ACSQ_TOT", "CRSQ_RUM", "ATS_SC", "ATS_G", "ATS_HS")


analysis_df <- speech_visit %>%      # still the most recent wave <= speech, within 365d
  add_sameday("CDI") %>%        # speech day only
  add_sameday("MASC") %>%       # speech day only
  relocate(ID, Date)

for (i in IVs) {analysis_df <- add_prior(analysis_df, i)}
```

Measurement timing

Each variable is drawn from a different point relative to the speech. `add_prior()` takes the most recent administration at or before the TSST (within 365 days); `add_sameday()` takes only the value recorded at the TSST visit itself.

```
timing_vars <- tribble(
  ~Variable,  ~Role,        ~Rule,
  "CHS_TOT",  "Predictor",  "add_prior() - most recent wave <= speech, within 365d",
  "CDI",      "Covariate",  "add_sameday() - speech visit only",
  "MASC",     "not used",   "add_sameday() - speech visit only"
)

timing_summary <- timing_vars %>%
  rowwise() %>%
  mutate(
    n_available = sum(!is.na(analysis_df[[Variable]])),
    same_day    = if (paste0(Variable, "_gap") %in% names(analysis_df))
                    sum(analysis_df[[paste0(Variable, "_gap")]] == 0, na.rm = TRUE)
                  else sum(!is.na(analysis_df[[Variable]])),
    earlier     = n_available - same_day,
    `median days before` = if (paste0(Variable, "_gap") %in% names(analysis_df)) {
        g <- analysis_df[[paste0(Variable, "_gap")]]; g <- g[!is.na(g) & g > 0]
        if (length(g)) median(g) else NA_real_
      } else NA_real_,
```

```
    `max days before` = if (paste0(Variable, "_gap") %in% names(analysis_df)) {
        g <- analysis_df[[paste0(Variable, "_gap")]]; g <- g[!is.na(g) & g > 0]
        if (length(g)) max(g) else NA_real_
      } else NA_real_
  ) %>%
  ungroup()

kable(timing_summary,
      caption = paste0("Measurement timing relative to the TSST speech. ",
                       "Models are fitted on n = ",
                       sum(complete.cases(analysis_df[, c("CHS_TOT", norms,
                             c("age","female","CDI","CHS_TOT_gap","n_words"))])),
                       " participants with complete data."))
```

Table 1: Measurement timing relative to the TSST speech. Models are fitted on n = 66 participants with complete data.

Variable	Role	Rule	n_available	same_day	earlier	median days before	max days before
CHS_TOT	Predictor	add_prior() - most recent wave <= speech, within 365d	68	39	29	214	362
CDI	Covariate	add_sameday() - speech visit only	88	88	0	NA	NA
MASC	not used	add_sameday() - speech visit only	88	88	0	NA	NA

significant results

```
kable(results_regression_sig)
```

model_num	IV	DV	model_type	beta	p	adj_p
1	CHS_TOT	emo_valence_b24	lm	-0.0779023	0.0090256	0.0362329
3	CHS_TOT	emo_anxiety	lm	0.0050752	0.0290269	0.0509151
4	CHS_TOT	emo_sadness	lm	0.0061921	0.0136949	0.0410847
5	CHS_TOT	emo_guilt	lm	0.0054741	0.0297005	0.0509151
6	CHS_TOT	lex_AoA	lm	-0.1131462	0.0257170	0.0509151
7	CHS_TOT	lex_freqlg10	lm	0.0852364	0.0036846	0.0362329
9	CHS_TOT	phon_n_lett	lm	-0.1001185	0.0436815	0.0614557
10	CHS_TOT	phon_nsyll	lm	-0.0368426	0.0479797	0.0614557
12	CHS_TOT	sem_neighbors	lm	135.8188191	0.0090582	0.0362329

significant results post correction

```
sig_corrected <- results_regression_sig %>%
  filter(adj_p < .05)
sig_corrected %>% kable()
```

model_num	IV	DV	model_type	beta	p	adj_p
1	CHS_TOT	emo_valence_b24	lm	-0.0779023	0.0090256	0.0362329

model_num	IV	DV	model_type	beta	p	adj_p
4	CHS_TOT	emo_sadness	lm	0.0061921	0.0136949	0.0410847
7	CHS_TOT	lex_freqlg10	lm	0.0852364	0.0036846	0.0362329
12	CHS_TOT	sem_neighbors	lm	135.8188191	0.0090582	0.0362329

```
sig_model_nums <- sig_corrected$model_num
```

all results:

```
kable(results_regression_summary)
```

IV	DV	model_type	beta	p
CHS_TOT	emo_valence_b24	lm	-0.0779023	0.0090256
CHS_TOT	emo_anger	lm	0.0023259	0.2770905
CHS_TOT	emo_anxiety	lm	0.0050752	0.0290269
CHS_TOT	emo_sadness	lm	0.0061921	0.0136949
CHS_TOT	emo_guilt	lm	0.0054741	0.0297005
CHS_TOT	lex_AoA	lm	-0.1131462	0.0257170
CHS_TOT	lex_freqlg10	lm	0.0852364	0.0036846
CHS_TOT	lex_n_morphemes	lm	-0.0125323	0.2800064
CHS_TOT	phon_n_lett	lm	-0.1001185	0.0436815
CHS_TOT	phon_nsyll	lm	-0.0368426	0.0479797
CHS_TOT	sem_diversity	lm	0.0138384	0.0512131
CHS_TOT	sem_neighbors	lm	135.8188191	0.0090582

```
# results_regression_list %>% map(~ summary(.x))
#
# results_regression_list %>%
#   map(~ resid_panel(.x))
#

results_regression_list %>%
  keep_at(sig_model_nums) %>%
  map(~ list(
    summary = summary(.x),
    residuals = resid_panel(.x)
  ))
```

```
library(kableExtra)

LAB <- c(emo_valence_b24 = "Valence", emo_anger = "Anger", emo_anxiety = "Anxiety",
         emo_sadness = "Sadness", emo_guilt = "Guilt",
         lex_AoA = "Age of acquisition", lex_freqlg10 = "Word frequency (log10)",
         lex_n_morphemes = "Number of morphemes", phon_n_lett = "Word length (letters)",
         phon_nsyll = "Syllables", sem_diversity = "Contextual diversity",
         sem_neighbors = "Neighbourhood density")
FAM <- setNames(c(rep("Emotion (H3)", length(emoDVs)),
                  rep("Lexical/phonological (H1)", length(lexDVs)),
                  rep("Semantic (H2)", length(semDVs))), DVs)
z  <- function(x, d = 3) sub("^(-?)0\\.", "\\1.", sprintf(paste0("%.", d, "f"), x))
```

```r
zp <- function(x) ifelse(x < .001, "< .001", ifelse(x > .999, "> .999", z(x)))

apa1 <- map2_dfr(results_regression_list, transpose(combo), function(m, sp) {
  DV <- sp[[2]]; s <- summary(m)
  co <- s$coefficients["CHS_TOT", ]; ci <- confint(m)["CHS_TOT", ]
  d  <- model.frame(m)
  r2r <- summary(lm(as.formula(paste(DV, "~", paste(covariates[[DV]], collapse = " + "))),
                    data = d))$r.squared
  tibble(Family = FAM[[DV]], Outcome = LAB[[DV]],
         b = co[1], SE = co[2], lo = ci[1], hi = ci[2],
         beta = co[1] * sd(d$CHS_TOT) / sd(d[[DV]]),
         t = co[3], p = co[4], sr2 = s$r.squared - r2r, R2 = s$r.squared)
}) %>% mutate(q = p.adjust(p, "BH")) %>%
  arrange(match(Family, c("Lexical/phonological (H1)", "Semantic (H2)", "Emotion (H3)")), p)

apa1 %>%
  transmute(Outcome,
            b = sprintf("%.3f", b), SE = sprintf("%.3f", SE),
            `95% CI` = sprintf("[%.3f, %.3f]", lo, hi),
            `$\\beta$` = z(beta), t = sprintf("%.2f", t),
            p = zp(p), q = zp(q), `$sr^2$` = z(sr2), `$R^2$` = z(R2)) %>%
  kable(booktabs = TRUE, align = "lrrcrrrrrr",
        caption = "Hopelessness predicting psycholinguistic features of TSST speech") %>%
  kable_styling(latex_options = c("hold_position", "scale_down")) %>%
  pack_rows(index = table(fct_inorder(apa1$Family))) %>%
  footnote(general_title = "Note.", general = paste0(
    "N = ", nobs(results_regression_list[[1]]), ", df = ",
    df.residual(results_regression_list[[1]]),
    " for every model. Each row is a separate regression controlling for age, sex, ",
    "same-day CDI, the hopelessness assessment gap, word count, and child race. ",
    "Hopelessness is standardized, so b is the change in raw outcome units per 1 SD ",
    "of hopelessness; beta is fully standardized. q values are Benjamini-Hochberg adjusted."),
    threeparttable = TRUE, footnote_as_chunk = FALSE)
```

```
## Warning in styling_latex_scale(out, table_info, "down"): Longtable cannot be
## resized.
```

Table 5: Hopelessness predicting psycholinguistic features of TSST speech

Outcome	b	SE	95% CI	β	t	p	q	sr^2	R^2
Lexical/phonological (H1)									
Word frequency (log10)	0.085	0.028	[0.029, 0.142]	.434	3.03	.004	.036	.132	.182
Age of acquisition	-0.113	0.049	[-0.212, -0.014]	-.339	-2.29	.026	.051	.080	.125
Word length (letters)	-0.100	0.049	[-0.197, -0.003]	-.307	-2.06	.044	.061	.066	.118
Syllables	-0.037	0.018	[-0.073, -0.000]	-.302	-2.02	.048	.061	.064	.108
Number of morphemes	-0.013	0.011	[-0.036, 0.010]	-.164	-1.09	.280	.280	.019	.097
Semantic (H2)									
Neighbourhood density	135.819	50.262	[35.171, 236.467]	.379	2.70	.009	.036	.100	.217
Contextual diversity	0.014	0.007	[-0.000, 0.028]	.293	1.99	.051	.061	.060	.135
Emotion (H3)									
Valence	-0.078	0.029	[-0.136, -0.020]	-.350	-2.70	.009	.036	.085	.333
Sadness	0.006	0.002	[0.001, 0.011]	.336	2.54	.014	.041	.079	.304

Anxiety	0.005	0.002	[0.001, 0.010]	.298	2.24	.029	.051	.062	.296
Guilt	0.005	0.002	[0.001, 0.010]	.301	2.23	.030	.051	.064	.272
Anger	0.002	0.002	[-0.002, 0.007]	.147	1.10	.277	.280	.015	.282

Note.
N = 66, df = 57 for every model. Each row is a separate regression controlling for age, sex, same-day CDI, the hopelessness assessment gap, word count, and child race. Hopelessness is standardized, so b is the change in raw outcome units per 1 SD of hopelessness; beta is fully standardized. q values are Benjamini-Hochberg adjusted.

```
CVLAB <- c(CHS_TOT = "Hopelessness (CHS)", age = "Age", female = "Sex",
           CDI = "Depressive symptoms (CDI)", CHS_TOT_gap = "Assessment gap (days)",
           n_words = "Word count")

apa2 <- map2_dfr(results_regression_list, transpose(combo), function(m, sp) {
  DV <- sp[[2]]; s <- summary(m)$coefficients; ci <- confint(m)
  tibble(Outcome = LAB[[DV]], Term = rownames(s), b = s[, 1], SE = s[, 2],
         lo = ci[, 1], hi = ci[, 2], t = s[, 3], p = s[, 4]) %>%
    filter(Term != "(Intercept)") %>%
    mutate(Term = case_when(
      Term %in% names(CVLAB) ~ unname(CVLAB[Term]),
      Term == "ChildRace2"   ~ "Race: African American vs Caucasian",
      Term == "ChildRace3"   ~ "Race: Biracial/Other vs Caucasian",
      TRUE ~ Term))
})

apa2 %>%
  transmute(Predictor = Term, b = sprintf("%.3f", b), SE = sprintf("%.3f", SE),
            `95% CI` = sprintf("[%.3f, %.3f]", lo, hi),
            t = sprintf("%.2f", t), p = zp(p)) %>%
  kable(booktabs = TRUE, longtable = TRUE, align = "lrrcrr",
        caption = "Full regression models for each outcome") %>%
  kable_styling(latex_options = c("repeat_header")) %>%
  pack_rows(index = table(fct_inorder(apa2$Outcome)))
```

```
combo %>%
  pmap(~ ggplot(data = analysis_df, mapping = aes(x = .data[[..1]], y = .data[[..2]])) +
         geom_point() +
         geom_smooth(method = "lm"))


sig_corrected %>%
  filter(adj_p < .05) %>%
  pmap(~ ggplot(data = analysis_df, mapping = aes(x = .data[[..2]], y = .data[[..3]])) +
         geom_point() +
         geom_smooth(method = "lm"))
```